

Bases de données documentaires et distribuées
Cours NFE04
Introduction 'a XQuery

Auteurs : Raphaël Fournier-S'niehotta, Philippe Rigaux, Nicolas Travers
prénom.nom@cnam.fr

Département d'informatique
Conservatoire National des Arts & Métiers, Paris, France

Plan de la présentation

- 1 Préliminaires
- 2 L'essentiel : la clause FLOWR

Modèle de données et expressions

XQuery travaille sur des **sequences**, notion très générale.

Pour ce cours : on considère un cas spécial de séquence, la **collection**, constituée de documents XML.

Une **expression** XQuery prend en entrée une collection et renvoie une collection.

Deux fonctions permettent d'accéder aux collections :

- `doc()` prend en entrée l'URI d'un document XML et renvoie une collection-singleton contenant **le nœud racine** du document.
- `collection()` prend en entrée l'URI d'une collection et renvoie une séquence composée **des nœuds racine** des documents de la collection.

XPath et au-delà

Toute expression XPath est une requête XQuery. Voici une expression XPath/XQuery recherchant tous les films parus en 2005.

```
collection('movies')/movie[year=2005]/title
```

Le résultat est un **sequence** de nœuds `title` ::

```
<title>A History of Violence</title>  
<title>Match Point</title>
```

Remarque

L'expression XPath est évaluée pour chaque **Document** de la séquence `collection('movies')`.

Constructeurs

XQuery permet la construction de nouveaux éléments, dont le contenu peut mêler des éléments littéraux, des valeurs, ou le résultat d'autres requêtes (composition!).

```
<titles>  
  {collection('movies')//title}  
</titles>
```

Remarque

Une expression *e* doit être encadrée par des accolades {} pour être reconnue.

Variables

Des variables peuvent être utilisées dans les expressions XQuery. Par exemple ;

```
<employee empid="{ $id } ">  
  <name>{ $name }</name>  
  { $job }  
  <deptno>{ $deptno }</deptno>  
  <salary>{ $SGMLspecialist+100000 }</salary>  
</employee>
```

où les variables \$id, \$name, \$job, \$deptno and \$SGMLspecialist sont instanciées (une valeur est associée à chaque variable).

Plan de la présentation

- 1 Préliminaires
- 2 L'essentiel : la clause FLOWR

Expressions FLWOR

- 4 clauses :
 - For (itération sur une séquence),
 - Let (définition ou instanciation d'une variable),
 - Where (prédicat),
 - Order by (tri du résultat), and
 - Return (constructeur de résultat).
- Dans l'esprit du SELECT-FROM-WHERE de SQL

Expressions FLWOR (suite)

Dans l'une de ses formes les plus simples, FLWOR est une alternative à XPath.

```
//actor[birth_date>=1960]/last_name
```

est équivalent à l'expression (requête) XQuery suivante :

```
let $year:=1960
for $a in doc("SpiderMan.xml")//actor
where $a/birth_date >= $year
return $a/last_name
```

Expressions FLWOR (suite)

Les clauses `for` et `let`.

Ces deux clausesinstancient des variables.

`for` instancie une variable en lui faisant prendre (successivement) les valeurs des items d'une séquence en entrée

Par exemple : `for $x in /company/employee`
instancie la variable `x` en lui faisant successivement prendre pour valeur **chacun** des éléments `employee` de la séquence issue de l'évaluation de `/company/employee`.

`let` instancie une variable en lui faisant prendre la valeur de la séquence complète

Par exemple : `let $x := /company/employee`
instancie la variable `x` en lui faisant prendre la valeur de **la séquence** issue de l'évaluation de `/company/employee`.

Expressions FLWOR (suite)

Instanciation d'une variable :

```
let $years:=(1960,1978)  
return $years
```

Parcours d'une séquence :

```
for $i in (1,2,3)  
return  
<chantons>  
  { $i } kilometre(s) a pieds,  
  Ca use, ca use  
  { $i } kilometre(s) a pieds,  
  Ca use les souliers  
</chantons>
```

Expressions FLWOR (suite)

La séquence peut être obtenue par évaluation d'une expression XQuery (ci-dessous XPath)

Instanciation d'une variable :

```
let $speechs := doc("hamlet.xml")//SPEECH  
return $speechs
```

Parcours d'une séquence :

```
for $speech in doc("hamlet.xml")//SPEECH  
return $speech
```

Pour travailler le résultat :

```
for $speech in doc("hamlet.xml")//SPEECH  
return $speech/SPEAKER
```

Expressions FLWOR (suite)

Un autre exemple :

```
let $born := ('born, born, born')
let $alive := (' to be alive')
for $i in (1,2,3,4,5,6)
  return
    <refrain numero="{ $i }">
      You see you were
      { $born } { $alive }
      You see you were
      { $born }
      born { $alive }
    </refrain>
```

Test

Que renvoie cette expression XQuery ?

Expressions FLWOR (suite)

Un autre exemple :

```
for $i in (-3,-2,-1,0,1,2,3)
let $j := $i + 1
return <suivant>{$j} est le nombre suivant {$i}</suivant>
```

Test

Que renvoie cette expression XQuery ?

Expressions FLWOR (suite)

La clause `where` est assez similaire à son synonyme SQL. Elle permet de filtrer un item en fonction d'une condition.

```
for $line in doc("hamlet.xml")//SPEECH/LINE
where contains($line,"To be, or not to be")
return $line/parent::SPEECH/parent::SCENE/TITLE
```

```
<TITLE>SCENE I. A room in the castle.</TITLE>
```

Expressions FLWOR (suite)

```
for $line in doc("hamlet.xml")//SPEECH/LINE
where contains($line,"heart")
return $line/parent::SPEECH/child::SPEAKER
```

Pour chaque élément (stocké dans \$line) de
doc("hamlet.xml")//SPEECH/LINE
satisfaisant contains(\$line,"heart")
retourner \$line/parent::SPEECH/child::SPEAKER

Expressions FLWOR (suite)

```
for $line in doc("hamlet.xml")//SPEECH/LINE
where contains($line,"heart")
return $line/parent::SPEECH/child::SPEAKER
```

```
<SPEAKER>FRANCISCO</SPEAKER>
<SPEAKER>KING CLAUDIUS</SPEAKER>
<SPEAKER>KING CLAUDIUS</SPEAKER>
<SPEAKER>KING CLAUDIUS</SPEAKER>
<SPEAKER>KING CLAUDIUS</SPEAKER>
<SPEAKER>KING CLAUDIUS</SPEAKER>
<SPEAKER>KING CLAUDIUS</SPEAKER>
<SPEAKER>HAMLET</SPEAKER>
<SPEAKER>LAERTES</SPEAKER>
<SPEAKER>OPHELIA</SPEAKER>
<SPEAKER>HAMLET</SPEAKER>
<SPEAKER>HAMLET</SPEAKER>
<SPEAKER>HAMLET</SPEAKER>
<SPEAKER>HAMLET</SPEAKER>
<SPEAKER>LORD POLONIUS</SPEAKER>
<SPEAKER>HAMLET</SPEAKER>
<SPEAKER>KING CLAUDIUS</SPEAKER>
...
```

Expressions FLWOR (suite)

where est comme en SQL, mais... attention, documents semi-structurés.

“Tous les films dirigés par W. Allen”

```
for $m in collection("movies")/movie
where $m/director/last_name="Allen"
return $m/title
```

Ca ressemble à une requête SQL. Attention aux différences suivantes :

- 1 si un chemin n'existe pas, pas d'erreur de typage : le résultat est **false**.
- 2 si un chemin retourne plusieurs nœuds, le résultat est vrai si **au moins** une des comparaisons retourne **true**.

“Tous les films avec Kirsten Dunst” (note : plusieurs acteurs dans un films !)

```
for $m in collection("movies")/movie
where $m/actor/last_name="Dunst"
return $m/title
```

La clause `return`

`return` est obligatoire et sert à construire un "item" résultat. Un tel item est construit pour chaque lien (*binding*) de la variables par la clause `for`.

```
for $m in collection("movies")/movie
let $d := $m/director
where $m/actor/last_name="Dunst"
return
  <div>
    {$m/title/text(), "directed by",
      $d/first_name/text(), $d/last_name/text()},
    with
    <ol>
      {for $a in $m/actor
        return <li>{$a/first_name, $a/last_name,
                    " as ", $a/role}</li>
      }
    </ol>
  </div>
```

Ordonner le résultat

La clause `order by` permet de spécifier l'ordre de tri du résultat.

```
for $p in doc("hamlet.xml")//PERSONA
order by $p
return $p
```

```
<PERSONA>A Captain.</PERSONA>
<PERSONA>A Gentleman</PERSONA>
<PERSONA>A Priest. </PERSONA>
<PERSONA>BERNARDO</PERSONA>
<PERSONA>CLAUDIUS, king of Denmark. </PERSONA>
<PERSONA>CORNELIUS</PERSONA>
<PERSONA>English Ambassadors. </PERSONA>
<PERSONA>FORTINBRAS, prince of Norway. </PERSONA>
<PERSONA>FRANCISCO, a soldier.</PERSONA>
<PERSONA>GERTRUDE, queen of Denmark, and mother to Hamlet. </PERSONA>
<PERSONA>GUILDENSTERN</PERSONA>
<PERSONA>Ghost of Hamlet s Father. </PERSONA>
<PERSONA>HAMLET, son to the late, and nephew to the present king.</PERSONA>
<PERSONA>HORATIO, friend to Hamlet.</PERSONA>
<PERSONA>LAERTES, son to Polonius.</PERSONA>
...
<PERSONA>VOLTIMAND</PERSONA>
```